

Can better handoffs make an RLM more reliable?

In a recursive language model (RLM), a main model can ask helper models to do smaller tasks. We call this exchange of work and answers a **handoff**.

Last time, we taught the model a basic routine.

Training examples taught it to inspect data and calculate with Python. Asking helpers reliably remained harder.

This time, we studied the handoffs.

Training improved a routine that uses helpers. Separately, we found that linking each helper answer to its input can substantially improve reading accuracy.

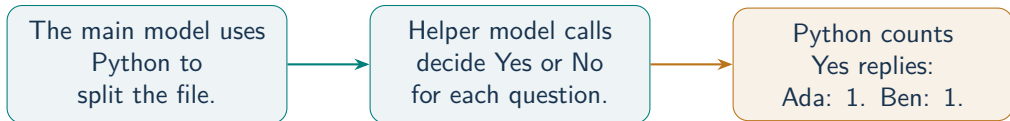
Alex Towell | Advisor discussion | Preliminary findings, 11 September 2026.

An RLM divides a large task into smaller steps.

The file stays in a Python workspace; the main model inspects pieces.

Task: Count each user's questions that ask for a location.

User	Question in the file	Helper: asks for a location?
Ada	Where is Oslo?	Yes
Ada	Who wrote Hamlet?	No
Ben	Where is Kyoto?	Yes



The hope: solve an unfamiliar whole by combining familiar smaller steps.

Invented example of a recursive language model (RLM). Helpers can divide work again; our tests mostly use one helper layer. The main model need not read the whole file at once.

We used supervised fine-tuning (SFT) to teach the main model a routine.

SFT updates a model to imitate the demonstrated action for the situation it sees. Here is a simplified example, after a helper has replied.

What the main model sees

Task: Count the questions asking for a location.

The Python list `types` holds the helper's replies:

```
location, human being, location
```

What it learns to produce

```
count = 0
for kind in types:
    if kind == "location":
        count += 1
print(count)
```

Python prints **2**. The training target is the action that calculates it, not just the number.

Illustrative data and shortened code, not a literal training record. Two copies started from the same earlier-trained model; each learned from 72 complete worked interactions. The helper stayed unchanged. [S1–S3]

Both separately trained copies solved more of the 72 test tasks.

Success required both the right final answer and the requested calculation using the helper's replies.



All 72 attempts count in each comparison. Attempts that timed out or lacked a verifiable final answer are not counted as successes.

Six attempts before SFT and two in the second copy timed out. One more pre-SFT attempt had no verifiable final answer. All versions faced the same tasks: familiar task types on eight newly selected input sets. [S3]

We tested whether names help link each helper answer to the right statement.

Each input item pairs a **text** with a **claim about that text**. One helper request contains several pairs; both texts and claims can vary.

Name	Text to read	Claim to check	Correct judgment
k7ab	Maya bought a red bike.	Maya bought a vehicle.	Supported
z2pm	Maya bought a red bike.	The bike is blue.	Contradicted
r9cx	A train arrived at noon.	The train was late.	Not enough information

Without matching names, replies are matched by order.

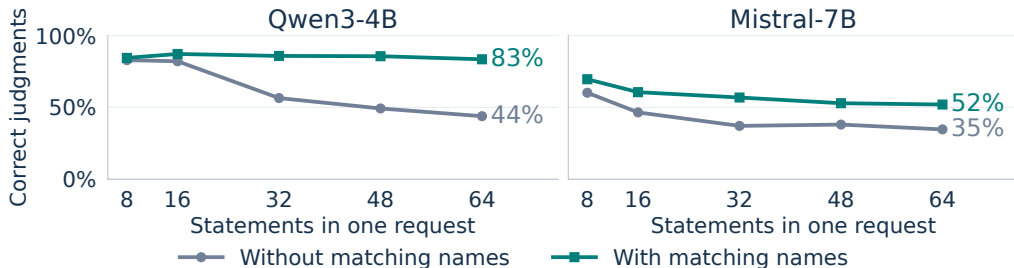
With matching names, each reply repeats its pair's name: k7ab: Supported.

Does this help the model judge many pairs accurately in one request?

Simplified examples; correct judgments are shown for explanation, not given to the model. The train's schedule is unstated. This separate experiment changes formatting, not training.

Matching names helped helpers judge many statements in one request.

Two language models judged statements about short texts, as in the bike example. Each request asked for several judgments. Neither model was retrained.



Same 16 input sets with and without names; 160 requests per model shown. Three Mistral replies broke the required format and count as wrong. These are helper judgments, not complete RLM solutions. [H8, H9]

Smaller requests were faster than large requests with matching names in this local test.

Each method judged the same 768 statements about short texts. We changed how many statements were sent to the helper together.

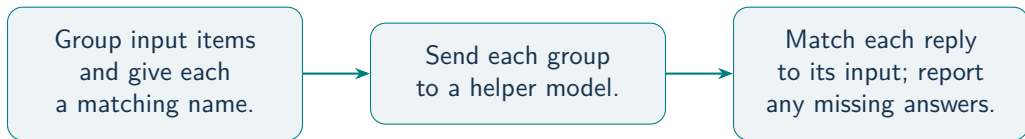
Statements per request	Correct judgments	Total time
48, without matching names	49%	19 seconds
48, with matching names	85%	91 seconds
16, without matching names	81%	19 seconds
1, without matching names	87%	31 seconds

Times cover all 768 statements, not one request. Sending one statement at a time was faster here, but repeated more input text.

We used Qwen3-4B on one A100, with up to four requests at once. Times add the measured test blocks and exclude server startup. All 848 requests had valid formatting. Other setups may give different speeds. [E1]

Next test: let the RLM program organize helper requests and match their answers.

The main model would still choose what to ask. A new program component would handle groups of input items and their replies.



Compare fixed group sizes with the current RLM and with one item per request, using the same tasks, models, and computing budget.

Success means a more reliable **final calculation**, not just better helper answers.

This is a proposal, not a proven improvement to complete solutions. Matching names is standard and cannot ensure correct reading. Learning when to change group size comes later.

We have a promising handoff result and a focused next research question.

What we know so far

Training examples helped the main model use helpers and calculate. Matching names improved the helpers' reading judgments without retraining.

What we want to establish next

Can the proposed program change improve the final answer at an acceptable cost in time and computation?

Would this be a worthwhile next study? What realistic task would make the test convincing?

The larger goal remains learning how to divide unfamiliar problems into useful smaller ones. Today's evidence is narrower. End of the main talk; the remaining pages are optional discussion material.

Example: the helper identifies question types, and Python adds the relevant points.

The points below are assigned test values, not confidence scores.

Add each user's points only for questions asking for a location. How many users have a total above 5?

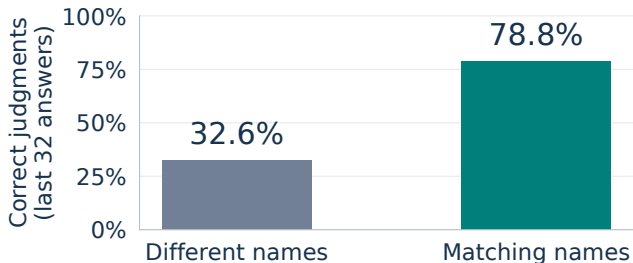
User	Question	Points	Asks for a location?
Ada	Where is Oslo?	4	Yes
Ada	What is the capital of Peru?	3	Yes
Ben	Who wrote Hamlet?	6	No
Ben	Where is Kyoto?	2	Yes

Ada: $4 + 3 = 7$. Ben: 2. Only Ada exceeds 5, so the answer is **1 user**.

Invented example, not a quoted training demonstration.

Using the same name on a statement and its answer helped more than using different names.

The helper judged statements about short texts. Answer order specified which statement to judge; both versions also supplied arbitrary names.



Input name → answer name

Different:

k7ab → z2pm

Matching:

k7ab → k7ab

Matching helped in all 16 input sets, even though both versions had valid formatting.

Each bar scores 3,072 judgments: the last 32 in each 48-statement request. Qwen3-4B used different input sets from slide 6. This shows behavior, not why it happens inside the model. [H6]

Reward training did not improve the final-answer count in this trial.

Reward training lets the model try solutions, then uses scores to adjust what it learns. We tested two reward rules from the same trained model.

Before reward
training

57

out of 72 questions.

Correct-answer
reward

55

out of 72 questions.

Answer reward
plus agreement check

54

out of 72 questions.

The extra check rewarded agreement with Python's calculation from the helper replies. The numbers above count only correct final answers.

Each run attempted 576 practice solutions before testing. The middle result has two unknown outcomes, so its total could be 55–57. We have not finished checking the calculations. This trial does not rule out reward training. [R3]

Training also helped with new combinations of familiar steps.

Example of a new combination

Find users who asked for a location, such as “Where is Oslo?” Then count *their other questions* asking for a number, such as “How many moons does Mars have?”

Before worked-example training

2 / 72

correct answers and calculations.

After worked-example training

29 / 72

correct answers and calculations.

The instructions supplied these steps. Training helped the model carry out a new combination, not invent the plan.

The displayed totals count confirmed successes. Eleven results before training and one after training are unknown. We reused eight input sets, but these combinations were absent from the training examples. [S4]

Misleading names can draw an answer toward the wrong input.

The helper judged statements about short texts, using misleading answer names.

Example of conflicting instructions

“Answer the statement in row 7, but put its answer beside the name row8.” Row 8 is a different input in the same request.

Name of another input,
such as row8

39%

of judgments were correct.

Unrelated name,
such as k7ab

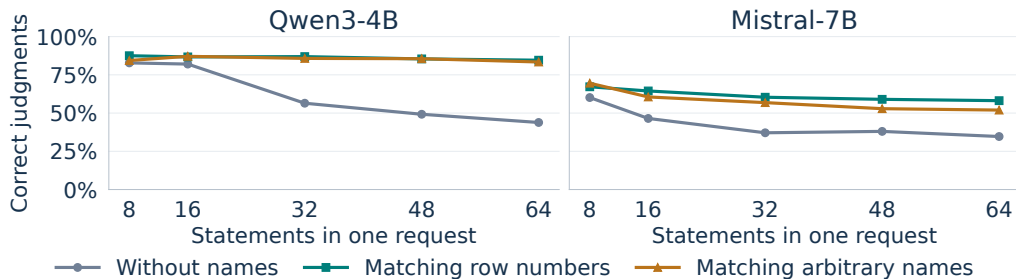
80%

of judgments were correct.

Invented illustration of a deliberate stress test. Software fixed names and order. Qwen3-4B; 16 groups of 48 statements. This measures output behavior, not attention. [H1]

Both row numbers and arbitrary names helped when helpers judged many statements together.

Two language models judged statements about short texts. We linked each statement to its answer using row numbers (1, 2, ...) or arbitrary names (k7ab, z2pm, ...).



Same 16 input sets; 240 requests per model. Three Mistral arbitrary-name responses had invalid formatting and count as wrong. This complicates comparing the two naming styles. Helper judgments, not complete RLM solutions. [H8, H9]